

In our experience, there are three main areas of the Enigma project that cause the most confusion in students: understanding how the internal rotor wires are represented in code, making sense of how to go through these rotors backwards, and interacting with the MCV pattern. The problems on this week’s section are designed to help you work through each of these areas of conceptual difficulty.

1. Tracing the Enigma Rotors

Figure 1 below shows an image of one of the Enigma rotors from two different perspectives: an edge-on view of the rotor along its axis and the “unrolled” view that shows the connections between the two sides of the rotors. The colored line in the figure shows that, in this position with the progress equal to its initial value of 0, position 0 on the right side of the rotor is connected to position 4 on the left side, which corresponds to a wiring offset of 4 (and would map **A** to **E**).

Use the diagram in Figure 1 to answer the following questions:

- With this current progress of 0, where does a signal starting at position 1 (**B**) on the right go on the left?
- When the rotor advances to progress 1, where does the signal from position 9 (**J**) go?
- When the rotor advances 3 more times to progress 4, where does a signal originating from position 11 (**L**) go?

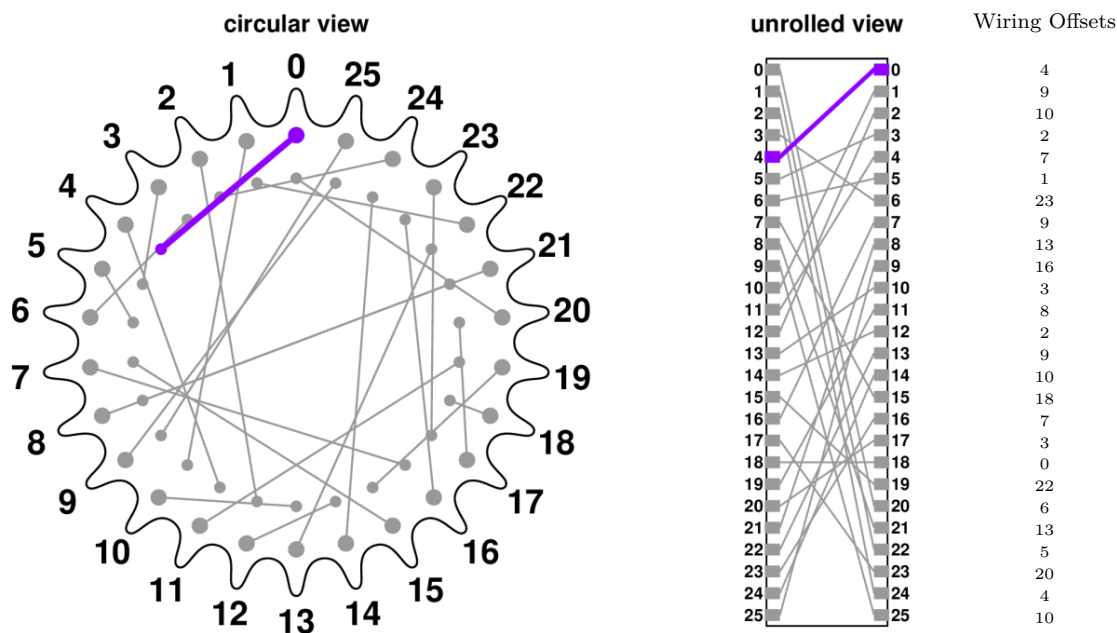


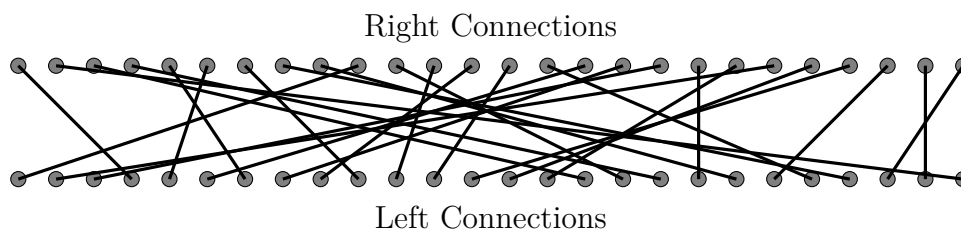
Figure 1: Two views of an Enigma rotor.

2. **Inverting the Wiring Offsets** Most letter-substitution ciphers require the sender and receiver to use different keys: one to encrypt the message and one to decrypt the message when it reaches its destination. Since each rotor in Enigma serves as its own letter-substitution cipher, this is no different. While the wiring offsets tell us what value on the right maps to what value on the left, in order to go the other direction we need to generate a new list of wiring offsets.

This is most easily illustrated by an example. Suppose that the right-to-left wiring offsets are given by the list:

[3, 24, 13, 14, 2, 25, 3, 15, 11, 17, 6, 25, 22, 24, 7, 16, 17, 11, 0, 21, 7, 18, 16, 23, 0, 24]

which could be visualized as:



The easiest way to create the inverse wiring offset list is to look at each of the original offsets, to see where they point. That will represent the index that we want to modify in the reversed offset list. For example, in the above list of offsets, the first offset is 3. As such, if we go to index-3 in our reversed offset list, we know the offset must be -3. Because if you entered from the left at index-3, you end up at index-0.

While negative offsets are ok, so far all wiring offsets have been positive. The easiest way to fix this is to just take the negative offset and then modulo/remainder by 26. So in the above case, $-3 \% 26$ would give 3.

If you loop over all the indexes and populate the reversed wiring offset list one value at a time, you'll generate the full reversed wiring offset list! And this is what you could use in the Enigma machine when the signal passes from left-to-right through the machine. Alternatively, it could be used to decode a signal that had only passed through a single rotor.

3. The last element of Enigma that can cause students to stumble is how the different parts of the program are communicating. In the MCV pattern, the *Controller* and *View* are responsible for consulting with the *Model* for what they should be doing at different times. The *Controller* will notify the *Model* whenever new and interest events occur, and the *View* will check in with the *Model* to get information from time to time about what it should be displaying. All of these interactions happen through the *View* or *Controller* calling particular methods defined inside the `EnigmaModel` class.

As such, interacting with the controller or view becomes a question of utilizing or defining the correct methods within the `EnigmaModel` class. **Do not rename these methods,**

or the already build *View* and *Controller* won't know how to talk to your model! Instead, you just need to update the code within the methods to accomplish the tasks that you want.

To get you going, there are three methods defined that control what is displayed in different parts of the screen:

- `is_key_pressed` controls if a key should light up as red or not, usually corresponding to it actively being pressed.
- `is_lamp_on` controls if a particular lamp is lit up or not, usually indicating what letter a key encrypts into
- `get_rotor_letter` controls what letter is displayed for each of the three rotors, which usually corresponds to the current progress of that rotor

In this problem, we just want to practice adding code to each of those methods to accomplish a particular task. Here, we want to recreate the exact situation seen in the screenshot below.



Figure 2: The desired output after completing this problem. Note that this is nonsensical output for how the Enigma machine will actually run, but it gets you familiar with how to update these various elements.

Note that because of when the model updates the view, you may need to click on of the keys after running the program to see all your changes take place.